AI TEST CASE GENERATOR

23CS55C - ARTIFICIAL INTELLIGENCE

MICRO PROJECT REPORT

Submitted by:

MOHAMED ABUBACKER SIDDIQ H - 2312060

MOHAMMED HAJI MEERAN - 2312076

Course Instructor

Ms. Kanthimathi AP CSE

Innovation and Problem statement (10 Marks)	Implementation & Results (10 Marks)	Presentation and Documentation (10 Marks)	Viva (10 Marks)	Total (40 Marks)

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION

In modern software development, Quality Assurance (QA) is paramount for delivering robust and reliable applications. However, the manual creation of test cases from requirements and user stories is notoriously time-consuming, repetitive, and often prone to human error and omissions, leading to gaps in test coverage.

The **Quality Assurance Test Case Generation Agent** project addresses this critical bottleneck by introducing an AI-driven solution designed to automate and streamline the test writing process. Leveraging the power of a **Large Language Model (LLM)**, this agent intelligently interprets application requirements and user stories to produce well-structured and comprehensive functional test cases.

Hosted locally via **Ollama** and employing models like **Mistral**, the agent provides a user-friendly interface built with **Streamlit**. QA teams can easily input requirement descriptions (or upload text files), and the system will automatically generate detailed test scenarios, including pre-conditions, numbered steps, expected results, priority, and test type. A core feature of the agent is its support for **iterative refinement**, allowing users to review, edit, and expand the generated JSON output, ensuring accuracy and comprehensive coverage for positive, negative, and edge-case scenarios.

Ultimately, this agent aims to significantly accelerate the QA cycle, reduce manual effort, minimize human error, and enhance the overall quality and efficiency of software testing by transforming textual requirements into actionable, exportable test case artifacts in both JSON and CSV formats.

CHAPTER 2

PROBLEM STATEMENT

QA teams play a vital role in ensuring the quality and reliability of software systems by developing comprehensive test cases based on application requirements and user stories. However, the traditional manual process of creating these test cases poses significant challenges. It is an extremely time-consuming and labor-intensive activity that requires deep understanding, meticulous attention to detail, and continuous collaboration among team members. Despite these efforts, manual test case generation remains prone to human errors, inconsistencies, and unintentional omissions, which can result in incomplete test coverage. Such gaps in coverage often lead to undetected defects, impacting the overall functionality, performance, and user experience of the software. As software projects grow in complexity and scale, the limitations of manual test creation become more pronounced, making it difficult for QA teams to maintain efficiency and accuracy under tight deadlines. This highlights the urgent need for an automated, intelligent solution capable of interpreting requirement documents and user stories to generate structured, accurate, and comprehensive test cases efficiently, thereby improving the speed, consistency, and quality of the software testing process.

CHAPTER 3

MODULE DESCRIPTION

1. **User Interaction & Presentation Module:**
    - **Responsibility:** Manages all user-facing aspects, including input collection, output display, and user interaction.
    - **Components:** Primarily streamlit_app.py.
    - **Functionality:** Provides the intuitive web interface for users to input requirements (text area or file upload), configure the LLM model, and provide additional generation instructions. It's also responsible for rendering the generated test cases in a user-friendly tabular format, displaying the raw or editable JSON output for refinement, and handling the trigger for generation, refinement, and data export actions. This module serves as the orchestrator for user workflows.

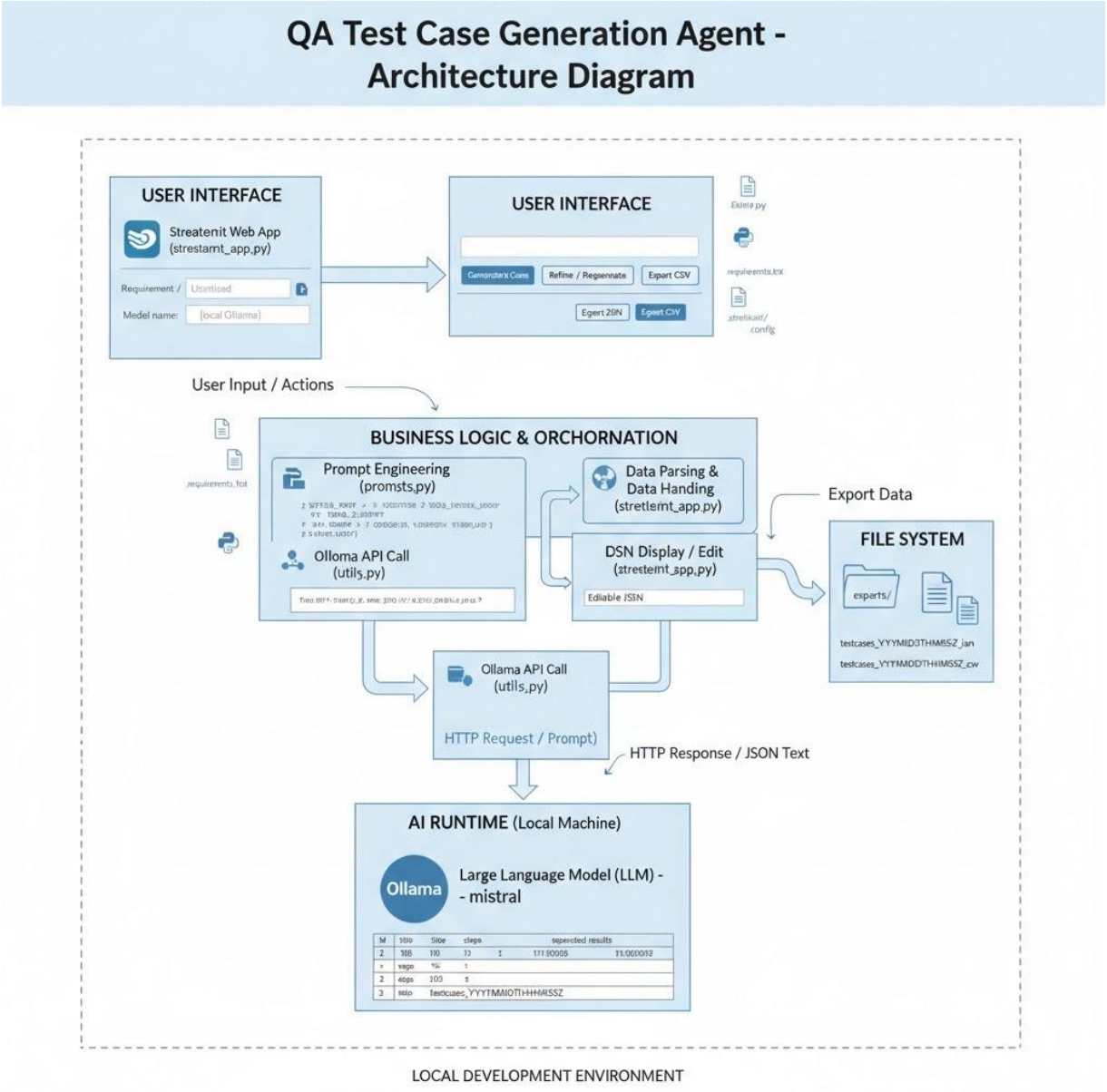
2. **AI Generation & Prompt Management Module:**
    - **Responsibility:** Encapsulates the core intelligence of the agent, focusing on preparing instructions for the LLM and communicating with it.
    - **Components:** prompts.py and the call_ollama function within utils.py.
    - **Functionality:** Defines the robust SYSTEM_PROMPT and GENERATION_TEMPLATE to guide the LLM's behavior and output format. It dynamically constructs comprehensive prompts by combining these templates with user-provided requirements and instructions. This module then handles the communication with the local Ollama server, sending the structured prompt to the chosen LLM (e.g., Mistral) and retrieving its raw text response.

3. **Data Transformation & Persistence Module:**
    - **Responsibility:** Processes the raw output from the AI, structures it for usability, and handles all data export functionalities.

- **Components:** safe_parse_json_array, testcases_to_dataframe, export_testcases_json, and export_testcases_csv functions within utils.py.
- **Functionality:** Takes the raw text output received from the LLM and intelligently parses it to extract a valid JSON array of test cases, even if the LLM includes extraneous text. It then transforms this structured data into a Pandas DataFrame for easy display and manipulation within the UI. Finally, this module manages saving the generated test cases into industry-standard JSON and CSV formats within the dedicated exports/ directory, allowing for seamless integration with other tools or documentation.

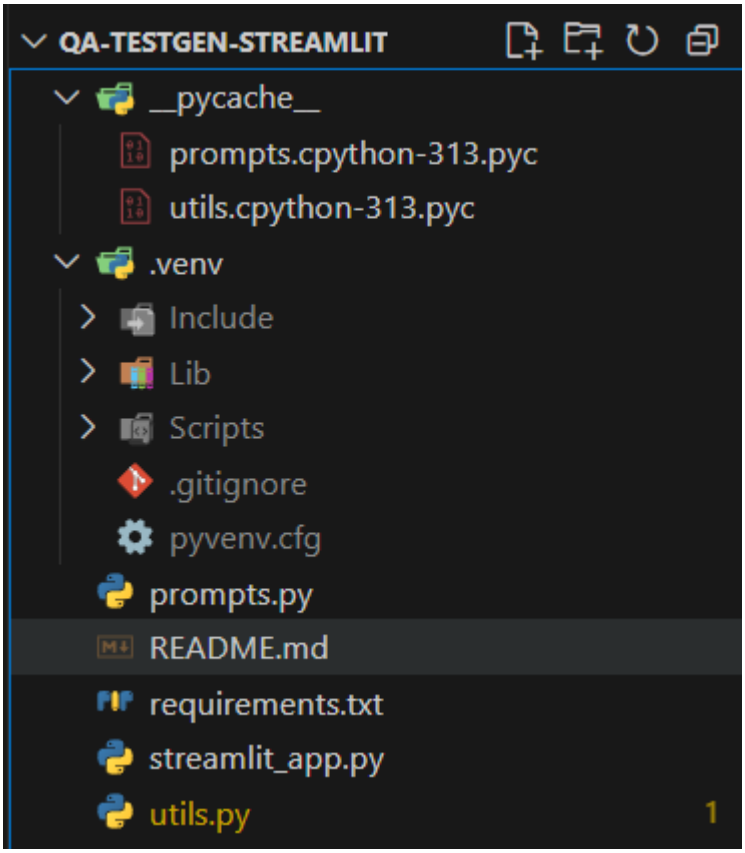
CHAPTER 4

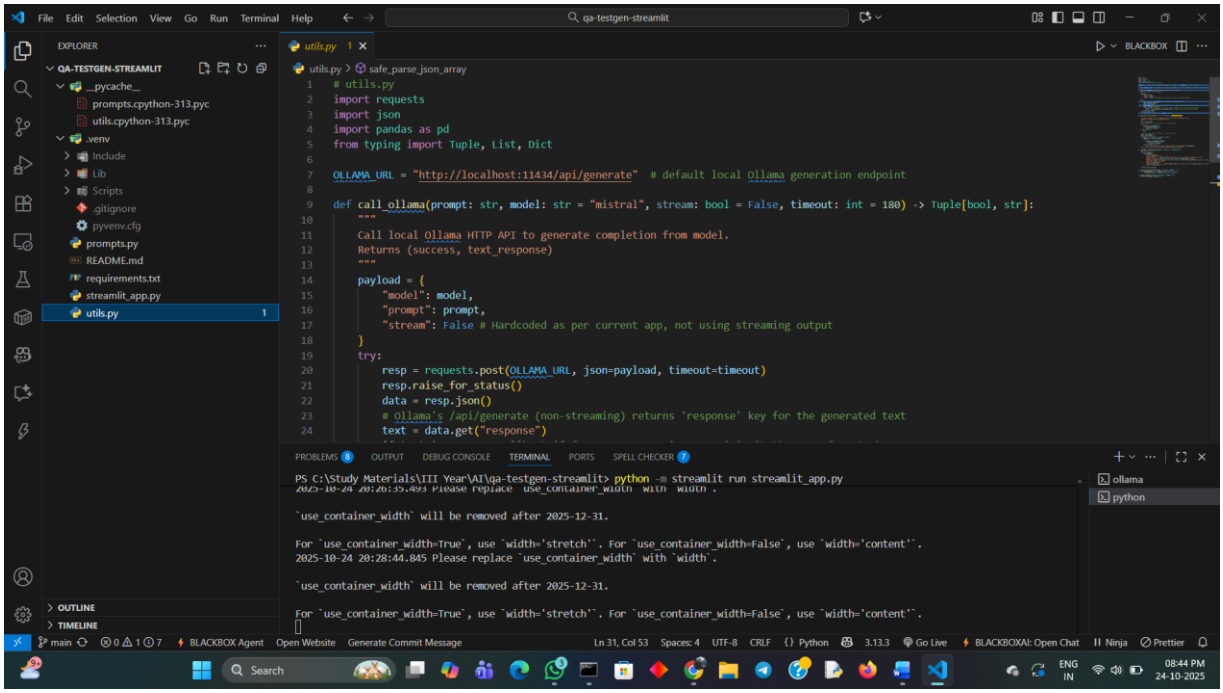
ARCHITECTURE DIAGRAM



CHAPTER 5

CODING AND IMPLEMENTATION





Streamlit_app.py:

```
import streamlit as st
from prompts import SYSTEM_PROMPT, GENERATION_TEMPLATE,
get_qa_generation_prompt # Added get_qa_generation_prompt
from utils import call_ollama, safe_parse_json_array, testcases_to_dataframe,
export_testcases_json, export_testcases_csv
import json
import os
from datetime import datetime

st.set_page_config(page_title="QA Test Case Generation Agent", layout="wide")

st.title("QA Test Case Generation Agent — Streamlit + Ollama Mistral")
st.markdown("Paste requirement(s) or a user story, then click **Generate Test
Cases**.")

# Input panel
with st.sidebar:
    st.header("Input")
    requirement_text = st.text_area("Requirement / User story", height=250,
placeholder="As a user, I want to ...")
    uploaded = st.file_uploader("Or upload a .txt file", type=["txt"])
    model_name = st.text_input("Model name (local Ollama)", value="mistral")
    if uploaded is not None:
        try:
            t = uploaded.read().decode("utf-8")
            # Only append if requirement_text already has content, otherwise replace
            if requirement_text.strip():
                requirement_text = requirement_text + "\n\n" + t
```

```
            else:
                requirement_text = t
        except Exception:
            st.error("Couldn't read uploaded file as text.")
    st.markdown("---")
    st.markdown("Advanced / Prompt tweaks")
    additional_instructions = st.text_area("Extra instructions for generator (optional)",
height=100,
                                           placeholder="e.g., Focus on security and boundary
conditions.")
    st.markdown("---")
    st.write("Ollama endpoint assumed: http://localhost:11434/api/generate")
    st.write("Make sure Ollama is running and model is pulled (e.g., `ollama pull
mistral`).")

# Buttons
col1, col2, col3 = st.columns([1,1,1])
with col1:
    generate_btn = st.button("Generate Test Cases")
with col2:
    refine_btn = st.button("Refine / Regenerate")
with col3:
    export_json_btn = st.button("Export JSON")
    export_csv_btn = st.button("Export CSV")

# Output area
if "last_testcases" not in st.session_state:
    st.session_state["last_testcases"] = None
if "last_raw" not in st.session_state:
```

```
    st.session_state["last_raw"] = ""

# Removed the 'build_prompt' function from here, as it's now in prompts.py

if generate_btn:
    if not requirement_text.strip():
        st.sidebar.error("Please provide a requirement or upload a text file.")
    else:
        st.info("Generating test cases — calling local Ollama model...")
        # Use the centralized prompt generation function
        prompt = get_qa_generation_prompt(requirement_text.strip(),
additional_instructions.strip())
        success, resp = call_ollama(prompt, model=model_name)
        if not success:
            st.error(resp)
        else:
            st.session_state["last_raw"] = resp
            ok, parsed_or_err = safe_parse_json_array(resp)
            if ok:
                st.session_state["last_testcases"] = parsed_or_err
                st.success(f"Parsed {len(parsed_or_err)} test case(s).")
            else:
                st.warning("Couldn't parse model output as JSON array. Showing raw output
for manual copy/edit.")
                st.session_state["last_testcases"] = None
                st.text_area("Raw model output", value=resp, height=300,
key="raw_output")

# Show last parsed testcases
```

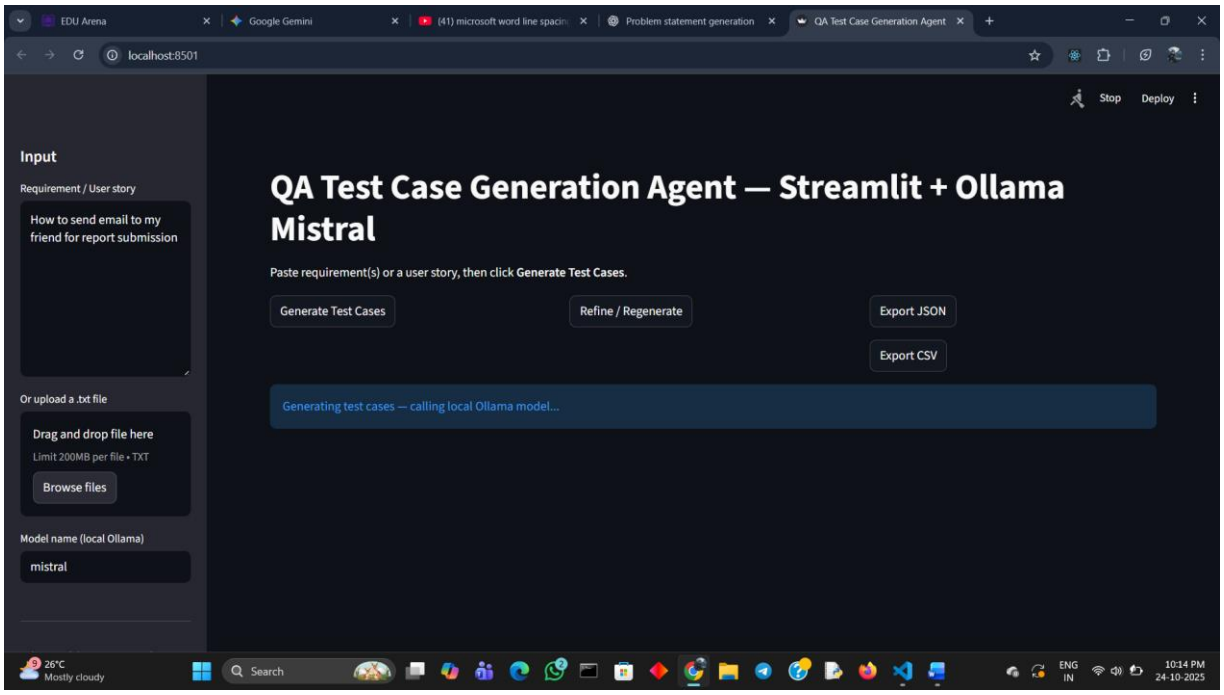
```
if st.session_state["last_testcases"]:
    st.subheader("Generated Test Cases (structured)")
    df = testcases_to_dataframe(st.session_state["last_testcases"])
    st.dataframe(df, use_container_width=True)

    # Show JSON in a text area for user edits / iterative refinement
    st.subheader("Edit JSON (for refinement)")
    editable_json = st.text_area("Editable JSON",
value=json.dumps(st.session_state["last_testcases"], indent=2, ensure_ascii=False),
height=300)

    # If user clicked refine: send the edited JSON back to model with instructions to
adjust/expand
    if refine_btn:
        st.info("Refining test cases — asking the model to expand/modify based on
edits...")
        # Use the user's edited JSON as instructions
        user_edit_instructions = "Please produce a corrected/expanded set of test cases
according to the JSON below. If you can improve coverage, add more test cases. Only
return a JSON array of test cases.\n\nUser JSON edits:\n" + editable_json
        prompt = SYSTEM_PROMPT + "\n\n" + user_edit_instructions #
SYSTEM_PROMPT is still valuable here
        success, resp = call_ollama(prompt, model=model_name)
        if not success:
            st.error(resp)
        else:
            st.session_state["last_raw"] = resp
            ok, parsed_or_err = safe_parse_json_array(resp)
            if ok:
                st.session_state["last_testcases"] = parsed_or_err
```

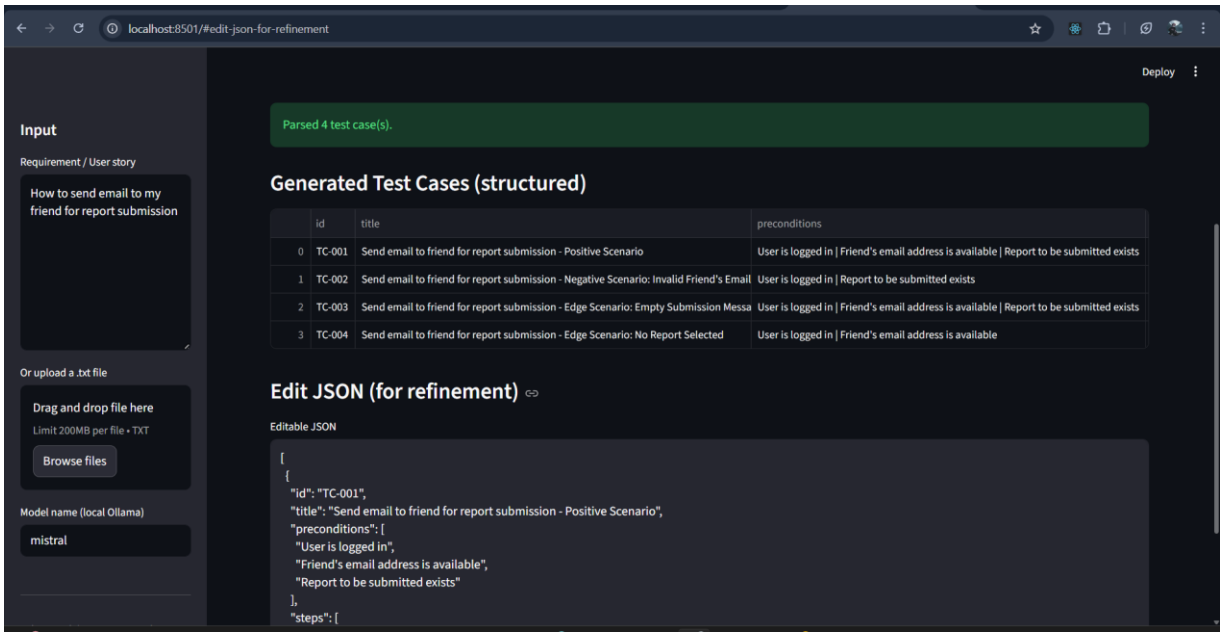
```
            st.success("Refinement parsed successfully.")
            df = testcases_to_dataframe(parsed_or_err)
            st.dataframe(df, use_container_width=True)
            # update editable area
            st.text_area("Editable JSON (new)", value=json.dumps(parsed_or_err,
indent=2, ensure_ascii=False), height=300, key="editable_after_refine")
        else:
            st.warning("Couldn't parse refined output as JSON array. Showing raw
output below.")
            st.text_area("Refined raw model output", value=resp, height=300)

    # Exports
    if export_json_btn:
        ts = st.session_state["last_testcases"]
        if ts:
            os.makedirs("exports", exist_ok=True)
            fname =
f"exports/testcases_{datetime.utcnow().strftime('%Y%m%dT%H%M%SZ')}.json"
            export_testcases_json(ts, fname)
            st.success(f"Saved JSON to `{fname}`")
            with open(fname, "r", encoding="utf-8") as f:
                st.download_button("Download JSON", f,
file_name=os.path.basename(fname), mime="application/json")
        else:
            st.error("No testcases to export.")

    if export_csv_btn:
        ts = st.session_state["last_testcases"]
        if ts:
            df = testcases_to_dataframe(ts)
```

```
            os.makedirs("exports", exist_ok=True)
            fname =
f"exports/testcases_{datetime.utcnow().strftime('%Y%m%dT%H%M%SZ')}.csv"
            export_testcases_csv(df, fname)
            st.success(f"Saved CSV to `{fname}`")
            with open(fname, "r", encoding="utf-8") as f:
                st.download_button("Download CSV", f,
file_name=os.path.basename(fname), mime="text/csv")
        else:
            st.error("No testcases to export.")
else:
    # If raw output exists show it for manual correction
    if st.session_state["last_raw"]:
        st.subheader("Last raw model output (not parsed to JSON)")
        st.text_area("Raw output", value=st.session_state["last_raw"], height=300)
```

CHAPTER 6

OUTPUT SCREENSHOTS





Input:

How to send email to my friend for report submission

Output:

Test Case Details

1. TC-001: Positive Scenario (Successful Submission)

- **Title**: Send email to friend for report submission - Positive Scenario
- **Priority**: High
- **Test Type**: Functional
- **Preconditions**: User is logged in, friend's email is available, and the report to be submitted exists.
- **Steps**: Navigate to the 'Report Submission' page, select the report, enter the friend's email, write a submission message, and click 'Submit Report'.
- **Expected Results**: The email is successfully sent with the report attached, and the friend receives it.
- **Acceptance Criteria Met**: Email is sent to the specified recipient; Report and submission message are included; Friend receives the email.

2. TC-002: Negative Scenario (Invalid Email)

- **Title**: Send email to friend for report submission - Negative Scenario: Invalid Friend's Email
- **Priority**: High
- **Test Type**: Functional
- **Preconditions**: User is logged in and the report to be submitted exists.
- Steps: Follow the positive flow but enter an invalid email address.
- **Expected Results**: An error message is displayed indicating the invalid email address, and no email is sent.
- **Acceptance Criteria Met**: No email is sent when entering an invalid email address; An error message is displayed.

3. TC-003: Edge Scenario (Empty Message)

- **Title**: Send email to friend for report submission - Edge Scenario: Empty Submission Message

- **Priority**: Medium
- **Test Type**: Functional
- **Preconditions**: All positive preconditions are met.
- **Steps**: Follow the positive flow but leave the submission message field empty.
- **Expected Results**: An error message is displayed indicating an empty submission message, and no email is sent.
- **Acceptance Criteria Met**: No email is sent when the submission message is empty; An error message is displayed.

4. TC-004: Edge Scenario (No Report Selected)

- **Title**: Send email to friend for report submission - Edge Scenario: No Report Selected
- **Priority**: Medium
- **Test Type**: Functional
- **Preconditions**: User is logged in and friend's email is available.
- **Steps**: Navigate to the page, enter the email and message, but do not select a report before clicking 'Submit Report'.
- **Expected Results**: An error message is displayed indicating no report was selected for submission, and no email is sent.
- **Acceptance Criteria Met**: No email is sent when no report is selected; An error message is displayed.

CHAPTER 7

RESULT

The **Quality Assurance Test Case Generation Agent** project successfully delivers a streamlined, AI-powered solution that revolutionizes test case creation. Its primary outcome is the **automated generation of structured, comprehensive functional test cases** directly from application requirements and user stories, with each generated test case offering detailed elements such as a unique ID, clear title, necessary preconditions, numbered steps, precise expected results, priority, test type, and relevant acceptance criteria. This automation significantly **accelerates the QA cycle** by drastically reducing the manual effort and time traditionally spent on test writing, while simultaneously **enhancing test quality and coverage** through the LLM's comprehensive interpretation of requirements, covering positive, negative, and edge-case scenarios. Furthermore, the system supports **iterative refinement**, allowing QA engineers to interactively modify and expand the generated JSON output to incorporate human expertise and project-specific nuances. Finally, all results are presented in a clear, tabular format within the Streamlit UI and are readily **exportable to universally compatible JSON and CSV file formats**, facilitating seamless integration into existing QA workflows. In essence, the project yields a more efficient, accurate, and scalable approach to test case authoring, empowering QA teams to allocate their focus to higher-level strategic testing.